Decisiones de Diseño

Documentación #3 — Proyecto Final BD III

Sistema de Gestión Hotelera · PostgreSQL + MongoDB

Este documento justifica las decisiones de diseño tomadas en ambos motores: criterios de modelado relacional en PostgreSQL, patrones de documento en MongoDB, y decisiones transversales sobre la distribución de responsabilidades entre schema, procedures y capa de aplicación.

Decisiones — PostgreSQL

Reparto de motores: PostgreSQL vs MongoDB

El núcleo transaccional (reservas, facturación, pagos) va a PostgreSQL porque exige ACID y bloqueo: el problema más crítico del negocio es que dos huéspedes no reserven la misma habitación en fechas solapadas, lo que requiere integridad estricta. Los datos flexibles y de estructura variable (perfiles, incidencias, reseñas) van a MongoDB porque cambian de forma entre huéspedes y crecen con cada estadía; en un esquema relacional exigirían muchas columnas nulables o tablas EAV.

CHECK en tablas

Se usaron restricciones CHECK para validar los dominios de estado en lugar de ENUM o una tabla de lookup, porque las opciones son fijas y conocidas de antemano. CHECK es más simple de mantener y el enunciado lo requería explícitamente.

SERIAL en lugar de UUID

Se eligió SERIAL para las claves primarias porque es más legible en consultas y reportes, y suficiente para el volumen del proyecto. UUID añadiría complejidad sin beneficio observable.

precio_unitario en consumo como snapshot

El precio unitario se guarda en el momento de la compra del servicio extra, no como referencia al precio actual del catálogo. Práctica estándar: una factura histórica no debe verse afectada por cambios futuros de precios.

fecha_checkin_real y fecha_checkout_real en reserva

Aunque el enunciado solo especifica fechas planificadas, se agregaron estos campos para registrar cuándo el huésped ingresó y salió realmente. Son necesarios para calcular la duración real de la estadía y para la auditoría operativa.

Factura nace como pendiente al crearse la reserva

La factura se crea junto con la reserva (no al momento del checkout) para poder registrar costos en caso de cancelación. Sin este campo, una reserva cancelada no tendría dónde cargar el costo de cancelación sin agregar columnas extra.

Daterange con intervalo semiabierto [) en el constraint

Se usa el intervalo semiabierto [fecha_inicio, fecha_fin) porque el día de salida no cuenta como ocupación: un huésped que sale el lunes no se solapa con otro que entra ese mismo lunes. Esto corresponde a la lógica matemática de dominios semiabiertos.

EXCLUDE constraint para el anti-solape de reservas

El exclusion constraint sobre la tabla reserva rechaza a nivel de motor cualquier insert con la misma habitación y fechas solapadas (para estados confirmada, en_curso o completada). Es la garantía real contra la doble venta: aunque dos operaciones simultáneas pasen la validación previa, el motor solo acepta una y lanza exclusion_violation a la otra.

Vistas normales: v_disponibilidad_inmediata y v_huespedes_hospedados

Ambas vistas son normales (no materializadas) porque sus datos deben ser en tiempo real. La disponibilidad y los huéspedes hospedados cambian con cada check-in y check-out; una vista materializada mostraría datos viejos a recepción.

Vistas materializadas: mv_ocupacion_mensual y mv_top_huespedes

Estas vistas se materializan porque son reportes gerenciales con agregaciones costosas que toleran hasta 24 horas de desfase. Se refrescan de noche con REFRESH MATERIALIZED VIEW cuando el tráfico a la base es bajo, evitando impactar las operaciones.

Atomicidad de las operaciones críticas

Los procedures sp_realizar_reserva (OC-01) y sp_checkout (OC-02) ejecutan varios pasos como una unidad todo-o-nada con bloques BEGIN/EXCEPTION. Si cualquier paso falla, se revierte todo: no queda una reserva sin su factura, ni una habitación en estado limpieza si el checkout no se completó.

Concurrencia con dos técnicas distintas

Cada operación maneja la concurrencia según si la fila ya existe. En OC-01 no se puede bloquear una fila que aún no existe, así que la garantía contra la doble venta simultánea la da el exclusion constraint. En OC-02 la reserva ya existe, por lo que se usa SELECT ... FOR UPDATE para bloquear la fila y evitar un doble checkout.

Reglas a nivel schema vs procedure

A nivel schema viven las reglas declarativas: FK, UNIQUE, CHECK y el exclusion constraint. A nivel procedure vive la lógica multipaso: cálculo de totales, transiciones de estado, costo de cancelación y manejo de concurrencia. Esta separación mantiene el schema limpio y concentra la lógica en un único lugar auditable.

Separación entre checkout y pago

Una reserva puede quedar completada (el huésped se fue) con la factura aún en pagada_parcial (debe saldo), caso habitual de huéspedes corporativos. Por eso el checkout (operación única e irreversible) y el cobro del saldo (sp_registrar_pago, operación independiente) están separados. La función fn_saldo_reserva calcula en todo momento cuánto queda por pagar.

Decisiones — MongoDB

Embebimiento en perfiles_huesped

Todos los subcampos del perfil (preferencias, programa de lealtad, alergias, idiomas) se embeben en un único documento porque un perfil siempre se lee completo. No hay beneficio en separarlo en documentos distintos.

Acciones embebidas en incidencias

Las acciones de seguimiento se embeben como un array dentro del documento de incidencia porque la relación es one-to-few y las acciones siempre se leen junto con la incidencia. Referenciarlas en una colección aparte añadiría un lookup innecesario.

Desnormalización del promedio en reseñas

El promedio de calificaciones se calcula una sola vez al insertar la reseña y se guarda como campo del documento. Sin esta desnormalización, habría que recalcularlo en cada consulta. La función normalizarResena garantiza su consistencia al momento de la inserción.

Embebimiento de los aspectos en reseñas

Los arrays aspectos_destacables y aspectos_mejorables son listas de strings normalizados (minúsculas, sin espacios extra, sin duplicados). La función JS normalizarResena garantiza la normalización antes de la inserción, lo que permite agrupar correctamente en el pipeline RC-09.

Validación en dos capas para categoría de incidencia

El campo categoria (queja, solicitud, observacion) y estado_resolucion se validan en dos capas: primero el enum de Mongoose, luego el validador $jsonSchema de la colección. La primera capa da un error legible; la segunda es la garantía a nivel de motor. Es el equivalente al CHECK de PostgreSQL para dominios en MongoDB.

Referencias lógicas en lugar de FK reales

Los campos huesped_id y reserva_id en las colecciones de MongoDB son enteros que referencian los IDs de PostgreSQL, pero no son foreign keys reales: MongoDB no puede forzar integridad referencial contra otro motor. La consistencia la garantiza la capa de aplicación, y los pipelines usan $match con $in para cruzar los datos.

Vocabulario abierto vs vocabulario cerrado

Se distinguen dos casos. Vocabulario abierto (aspectos_mejorables, aspectos_destacables): el huésped puede escribir cualquier valor, así que se normaliza el texto libre en lugar de restringirlo. Vocabulario cerrado (categoria, estado_resolucion): los valores posibles son fijos, lo que justifica usar enum en Mongoose y $jsonSchema, equivalente al CHECK de PostgreSQL.

Observaciones adicionales

Núcleo transaccional y relaciones

Las tablas huesped y habitacion convergen en reserva (la entidad crítica). De reserva cuelgan factura (1:1) y consumo (1:N). De factura cuelgan los registros de pago (1:N, para los pagos parciales). La relación M:N entre tipo_habitacion y comodidad se resuelve con la tabla puente tipo_habitacion_comodidad, conforme a 3FN.

Principio de embedding en MongoDB

La regla aplicada es: se embebe cuando los datos se leen siempre juntos y la subdocumentación es acotada (one-to-few). Se referencia cuando los datos crecen de forma ilimitada o ya existen en el otro motor.

Índices en MongoDB justificados por los reportes

perfiles_huesped: índice único sobre huesped_id (FK lógica hacia PostgreSQL). incidencias: índices sobre reserva_id y sobre categoria (RC-10 agrupa por categoría). resenas: índice único sobre reserva_id (una reseña por reserva) e índice sobre fecha (RC-08/09 agrupan por trimestre).

RC-11: demostración de la persistencia políglota

RC-11 es el punto de mayor integración entre motores: PostgreSQL calcula los id_huesped con más de 3 estadías completadas (consulta transaccional), y esa lista se pasa como filtro a un pipeline de MongoDB que agrega las preferencias, alergias e idiomas más comunes de ese grupo. Ningún motor por sí solo puede responder esta consulta.